

AutoTraq Development — Detailed Incident Report

BACKEND (API / Database / Services)

1. Backend Process Killed by Sub-Agent Tasks

What happened: During development, we used background AI sub-agents to build features in parallel (barcode system, part detail page, etc.). These sub-agents ran inside the same process tree as the backend server. When a sub-agent completed its task, the process cleanup sent a `SIGKILL` signal that cascaded up and killed the `tsx watch` process that was running the Express API server on port 3001. **How we discovered it:** Anson was actively using the app and suddenly got a white screen with "Failed to execute 'json' on 'Response': Unexpected end of JSON input" — the frontend was trying to call `/api/auth/me` but the backend was completely dead. No graceful error, no "server unavailable" message — just a raw JSON parse failure because the response was empty. **The fix:** Immediately ran `lsof -i :3001` to confirm the port was free, then restarted with `npm run dev`. Long-term, we created a root-level `package.json` using the `concurrently` package so both frontend and backend run in one terminal session with `npm run dev`, making restarts easier. We also became aware that sub-agent tasks needed to be isolated from the running server process. **Lesson learned:** Parallel development workflows need process isolation. A background build task should never share a process tree with a live server.

2. Port 3001 EADDRINUSE — Zombie Node Processes

What happened: After the backend crashed and restarted multiple times, orphaned Node.js processes accumulated. At one point, `ps aux | grep tsx` revealed **three separate `tsx watch` processes** all trying to bind to port 3001. Only one could hold the port — the others were zombies consuming memory but unable to serve requests. New restart attempts would fail immediately with `EADDRINUSE`. **How we discovered it:** Tried to restart the backend and got `Error: listen EADDRINUSE: address already in use :::3001`. The normal `lsof -ti :3001 | xargs kill -9` didn't work on the first try because the shell's `lsof` wasn't resolving to the right path on macOS. **The fix:** Used the full path `/usr/sbin/lsof -i :3001` to find the exact PID holding the port, then `kill -9 <pid>`. After clearing the zombie, the fresh `npm run dev` started cleanly. We identified that the issue was caused by the `tsx watch` spawning child processes that persisted after the parent died. **Lesson learned:** On macOS, always verify which `lsof` you're calling. Zombie process accumulation is a real risk when using file-watching dev servers with multiple restart triggers.

3. Wrong Startup Command — ts-node vs tsx

What happened: When the backend first went down, the initial restart attempt used `npx ts-node src/index.ts` instead of the project's configured `npm run dev` (which uses `tsx watch`). This immediately failed with `Cannot find module './routes/index.js'` because the codebase uses ESM-style `.js` extension imports in TypeScript files (e.g., `import routes from './routes/index.js'`). `ts-node` with its default CJS configuration couldn't resolve these — it looked for literal `.js` files that don't exist (only `.ts` files exist on disk).

How we discovered it: The error was immediate and clear — `MODULE_NOT_FOUND` with a require stack pointing at the import. **The fix:** Checked `package.json` scripts and found `"dev": "tsx watch src/index.ts"`. `tsx` handles ESM-style `.js` imports in TypeScript natively. Used the correct command going forward. **Lesson learned:** Always check the project's `package.json` scripts before improvising startup commands. TypeScript projects have specific toolchain requirements — `ts-node`, `tsx`, and `tsc + node` all behave differently with ESM imports.

4. Vehicle Search Returns Zero Results Despite 3,047 Vehicles

What happened: Anson typed "Ford" into the Vehicles search box and got "No vehicles found" despite the database containing 680+ Ford vehicles across all years. The search appeared completely broken. **Root cause analysis:** The search service used Prisma's `contains` filter on `make`, `model`, and `trim` fields. On MySQL, `contains` translates to `LIKE '%value%'` which is case-insensitive by default with the `utf8mb4_general_ci` collation. The query itself was correct. After extensive debugging — checking the database directly with raw MySQL queries (which returned Ford results), verifying the Vite proxy config, checking the API response format — the issue turned out to be that **Anson's locally running backend was still running old code** from before the search improvements were pushed. The `tsx watch` process had failed to pick up the git-pulled file changes because it was in a bad state from earlier crashes. **The fix:** Had Anson stop the backend completely (Ctrl+C), then restart fresh with `npm run dev`. The new process loaded the updated search service code and Ford immediately appeared. **Lesson learned:** File-watching dev servers can get into stale states after crashes and git operations. When search "doesn't work," always verify the running code matches the source files — a stale server is the most common cause.

5. Partial Year Search — "201" Returns Nothing

What happened: Anson typed "201" into the vehicle search expecting to see all vehicles from 2010 through 2019. Instead: zero results. The search only understood exact 4-digit years. **Root cause:** The search logic parsed the input as an integer and

checked `if (!isNaN(yearNum) && yearNum >= 1900 && yearNum <= 2100)`. The value `201` is a valid integer but it's not a valid year — it's 201 AD. So the year condition never matched, and "201" doesn't match any make, model, or trim as a text string either. **The fix:** Implemented partial year range matching. If the input is a number with fewer than 4 digits, pad it to create a range:

- "2" → 2000–2999 (too broad, but technically correct)
- "20" → 2000–2099
- "201" → 2010–2019
- "2014" → exact match 2014

Code: `const low = parseInt(token.padEnd(4, '0'), 10)` and `const high = parseInt(token.padEnd(4, '9'), 10)` → Prisma query `{ year: { gte: low, lte: high } }`. **Lesson learned:** Search UX needs to match how humans think. Nobody types a full year all at once — they type character by character and expect progressive filtering at each keystroke.

6. Multi-Word Search — "2014 ford" Shows All 2014 Vehicles

What happened: Anson typed "2014 ford" expecting to see only 2014 Ford vehicles. Instead, the results showed ALL 2014 vehicles — Buick, Cadillac, Chevrolet, everything. The "ford" part was completely ignored. **Root cause:** The search treated the entire input as a single string. `{ make: { contains: "2014 ford" } }` doesn't match anything because no make is literally named "2014 ford". The year condition matched `2014` from parsing the full string (JavaScript's `parseInt("2014 ford")` returns `2014` — it ignores trailing non-numeric characters). So only the year filter applied. **The fix:** Tokenized the search input by splitting on whitespace. For multi-word queries, each token generates its own set of OR conditions (could match year, make, model, or trim), then all tokens are combined with AND logic:

```
"2014 ford" → AND [
```

```
  OR [ year=2014, make LIKE '%2014%', model LIKE '%2014%' ],
```

```
  OR [ make LIKE '%ford%', model LIKE '%ford%', trim LIKE '%ford%' ]
```

```
]
```

This means "2014 ford mustang" correctly narrows to: year 2014 AND make contains "ford" AND model contains "mustang". **Lesson learned:** Multi-field search with mixed data types (integers and strings) requires careful tokenization. The AND/OR pattern — AND between tokens, OR across fields within each token — is the standard approach for faceted search.

7. SKU Decode Failure for Manual SKUs

What happened: On the Scan page, looking up a manually-created SKU like "BRK-001" returned "Could not decode SKU" in red. The part existed in the database but the SKU decoder couldn't parse it because it didn't follow the auto-generated format. **Root cause:** The SKU lookup service expected the format `{2-char make}-{3-char model}-{2-digit year}-{2-char system}{2-char component}` (e.g., `FD-MUS-17-ENBR`). A manual SKU like "BRK-001" has a different structure — 3 chars, dash, 3 chars — which fails the parser's length and segment checks. **The fix:** On the Scan page, changed the lookup flow: first search for the part by exact SKU match in the database. If found, navigate directly to the part detail page. Only attempt SKU decoding as a fallback for parts not in the database. Added clear messaging: "This SKU was decoded but no matching part exists in inventory" vs "No part found and could not decode SKU." **Lesson learned:** Don't assume all data follows the same format. The system needs to handle both auto-generated and manually-entered identifiers gracefully.

The “behind-the-scenes” part (the server)

- **A helper accidentally turned the server off.**

We used “robot helpers” to do jobs, but they were living in the same “house” as the server. When a helper finished, it cleaned up and accidentally shut off the whole server too. Then the app showed a white screen. We fixed it by restarting the server and making sure helpers don't share the same “house.”

- **The server door (port 3001) was stuck “already in use.”**

Sometimes the server crashed and left “ghost servers” behind. Those ghosts still acted like they were using the door, so the real server couldn't start. We found the ghost and kicked it out, then the server worked again.

- **We tried starting it the wrong way.**

We used the wrong start button, so the server couldn't find its parts and wouldn't run. We checked the right command and used that instead.

Searching for vehicles (cars) problems

- **Typing “Ford” said “no cars found,” even though there were lots.**

The search wasn't really broken — the computer was running an *old* version of the code. We restarted it, and it worked.

- **Typing “201” found nothing.**

People type slowly, like “2...0...1...” but the app only understood full years like “2014.” We taught it that “201” should mean “2010 to 2019.”

- **Typing “2014 ford” showed every 2014 car, not just Ford.**

The app only listened to the “2014” part and ignored “ford.” We fixed it so it listens to *each word* and narrows down correctly.

Scanning part codes (SKU) problems

- **Some codes were “handmade” and the app didn’t understand them.**

The app expected a special code style, but some parts had simple codes like “BRK-001.” We fixed it so it first checks the database for an exact match, and only tries to “decode” if it’s not found.